

EE-2003

Computer Organization and Assembly Language

Week 13 – Lecture 25

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Declaring Characters and Strings in Assembly
- 3 The "Hello World" Example
- 4 Subroutine: Clearing the Screen

Quick Review: What was in the Last Week?

- Display Memory
- ASCII Codes
- Writing Characters to the Screen

Declaring Characters and Strings in Assembly

- In Assembly Language, characters are stored as their **ASCII codes** in bytes.
- The assembler provides simplified syntax for declaring characters or strings:
 - `db 0x61, 0x62, 0x63`
 - `db 'a', 'b', 'c'`
 - `db 'abc'`
- All three forms are equivalent and store the same ASCII values.
- Strings are just a sequence of ASCII bytes stored in consecutive memory locations.

Overview: The "Hello World" Example

- Traditionally, the first program in any language prints **"Hello World"** on the screen.
- In Assembly, this involves detailed control over:
 - Memory management
 - Video memory access
 - Stack-based subroutine calls
- The program defines a string `'hello world'` and its length separately.
- Full code and explanation are available on Google Classroom:
<https://classroom.google.com/w/Nzc0NDg40TA0NjQw/t/all>

Subroutine: Clearing the Screen

- The `clrscr` subroutine clears the video memory.
- It uses:
 - Register `ES` to point to the video segment (`0xB800`)
 - Register `DI` as a pointer to each screen location
- Each screen character occupies a word (2 bytes): one for ASCII and one for attribute.
- Loop continues until the entire screen (4000 bytes) is cleared.

Subroutine: Printing a String

- The `printstr` subroutine displays any string on the screen.
- Parameters are passed through the stack:
 - Address of the string
 - Length of the string
- Uses:
 - SI to point to the string
 - DI to point to screen memory
 - CX to hold the string length
 - AH = attribute byte (white text on black)

Understanding the Print Mechanism

- Each character is displayed using:

$$[ES : DI] \leftarrow AX$$

where:

- AH = display attribute (color, intensity)
- AL = ASCII value of character
- After printing each character:
 - SI increases by 1 (next character)
 - DI increases by 2 (next screen position)
- The loop instruction automatically decrements CX and repeats until it reaches zero.

Program Execution Summary

- ① The screen is cleared using the `clrscr` subroutine.
- ② The address and length of the message are pushed on the stack.
- ③ `printstr` is called to display the string.
- ④ When executed, the message appears at the top-left of the screen.
- ⑤ The position and color attributes are currently hard-coded, but can be modified to display text anywhere.

For the complete program code and detailed explanation:

<https://classroom.google.com/w/Nzc0NDg4OTA0NjQw/t/all>

Thank You!

Any Questions?

Final Thoughts:

- Understanding strings and ASCII representation bridges software and hardware concepts.
- Direct screen control through video memory (0xB8000) reveals how data turns into visible text.
- Subroutines and stack management ensure modularity and reliability in assembly programs.
- The “Hello World” example illustrates the power of low-level programming.

“Assembly teaches us that behind every visible output lies a precise sequence of bytes — where logic meets hardware.”